Bean Beacon — Feature Improvement Ideas

A grounded list of user-facing feature gaps, pulled from actually reading the current backend and frontend code rather than guessing. Split into two groups: features that are **half-built and just need finishing**, and features that are **missing entirely**.

Half-built — quick wins

These already have real backend work behind them. Finishing them is mostly frontend wiring, not new architecture.

Favorites — fully built on the backend, completely unused on the frontend

The `Favorite` model, `favoriteController.ts` (get/add/remove), and `GET/POST/DELETE /api/favorites` routes all exist and work. But nothing in the frontend calls them — no API client methods, no "save" button wired up anywhere, no favorites list page. A user can't actually favorite a café today despite the feature being most of the way done.

To finish: add `favoritesApi` methods to `api.ts`, a toggleable heart/save icon on café cards, and a simple "My Favorites" view (could live on a new page, or as a filter tab on the main list).

Photo uploads — schema fields exist, nothing else does

The `Cafe` model has `photoPath` and `galleryPaths` fields, but there's no file upload endpoint, no `multer` (or equivalent) middleware, and no file input anywhere in `Manage.tsx`'s café submission form. Right now a user-submitted café can only ever have a photo if someone manually edits the database.

To finish: add an upload endpoint (local disk storage is fine to start; S3/Cloudinary later if needed), wire a file input into the submission form, and render the photo on `CafeProfile.tsx` and the café cards.

Missing entirely

Forgot password / reset flow

`authRoutes.ts` only has `register`, `login`, and `me` — there's no password reset path at all. If a user forgets their password, there's currently no way back in except re-registering with a different email. This also ties into **Étape 6 (security)** work from the course guide if it's graded — a reset flow is a natural place to demonstrate secure token handling (short-lived, single-use reset tokens, not emailing the password itself).

Scope: a `POST /api/auth/forgot-password` (issues a token, emails a link — needs an email service, even a free-tier one like Resend or a dev-only console-log fallback) and `POST /api/auth/reset-password` (verifies token, sets new password).

Owner notification when a submitted café is approved or rejected

Right now, after a user submits a café, their only way to find out if an admin approved or rejected it is to keep checking `Manage.tsx` themselves. There's no email, no in-app notification, nothing pushed to them.

Scope: simplest version is in-app only — a notification bell/badge that reads unread status changes on the user's own submissions (no new infrastructure needed, just a query against cafés the user owns where `status` changed recently). Email notification is a larger version of the same idea, once an email service exists (see password reset above — the two could share that infrastructure).

Distance sorting + pagination on the main café list

`GET /api/cafes/all` — the endpoint that actually powers the merged app's main list — sorts by `createdAt` only and hard-caps at 200 results with no `page` / `limit` params. The geospatial `$near` distance-sorting that exists on `getCafesByLocation` (the older OSM-import endpoint) isn't used there. So once there are more cafés than fit in that cap, or a user wants "closest to me" sorting on the real merged list, there's no way to get it.

Scope: add `page` / `limit` query params to `listAllCafes`, and when `lat` / `lng` are passed, sort by distance using the same `2dsphere` index that's already on the `Cafe` model — the index exists, it's just not used by this endpoint.

Suggested order

Favorites and photo uploads are the cheapest wins — the hard backend part is already done for favorites, and photo uploads is mostly plumbing. Forgot-password and owner notifications are both naturally bigger (need an email service), so doing them together once that infrastructure exists is efficient. Distance sorting + pagination is self-contained backend work with no new dependencies, so it can slot in anywhere.